Luke Wiley

Task 1: Data Preprocessing

A. For this task, I performed Principle Component Analysis on the data set to reduce the dimensionality from 14 to 3, to create a workable set of data that we can manipulate and draw conclusions from. I named the new dimensions NEW1, NEW2, and NEW3. Before this, I standardized the data. Below are the first five records in a data frame after performing this. This new data frame has shape (24000, 3).

	NEW1	NEW2	NEW3
0	2.030105	-0.908633	0.319742
1	-1.587407	0.185636	0.263450
2	-1.444669	0.415575	0.097748
3	-1.790438	-0.160231	0.025366
4	3.895848	-0.884615	-1.271163

B. Before K-Means, I had to normalize the data. For this, I adopted the MinMax scaling scheme, to get all values between 0 and 1. This data frame has the same three columns NEW1, NEW2, and NEW3. Below are the first five records in a data frame after performing this. This new data frame has shape (24000, 3).

	NEW1	NEW2	NEW3
0	0.125607	0.045850	0.164771
1	0.038353	0.059200	0.162904
2	0.041796	0.062006	0.157407
3	0.033456	0.054981	0.155005
4	0.170608	0.046143	0.111995

Task 2: K-Means Implementation

A. For the implementation of the K-means algorithm, I utilized the normalized PCA data with the attributes NEW1, NEW2, and NEW3 as mentioned before. Using the template given to us, I developed the helper functions rand_center, update_centroids, and converged, while also slightly altering the given kmeans function.
For the rand_center function, i utilized NumPy to create an array and select the random Centroids by index, and return the centroids themselves.

For update_centroids, I utilized the NumPy array to compute the distance of every point to the centroid in each cluster. I then average the points in the cluster to select a new centroid. If a cluster ever ends up with zero points, I reinitialize the centroid to a random point.
For converged, I check the changes of the centroids after each update_centroids call, to see if the old centroid and the new one are effectively the same. To prevent infinite loops, I added a convergence counter in this function that returns if the iterations every reach 100. I also slightly altered the K-means function to reset this counter with every call.

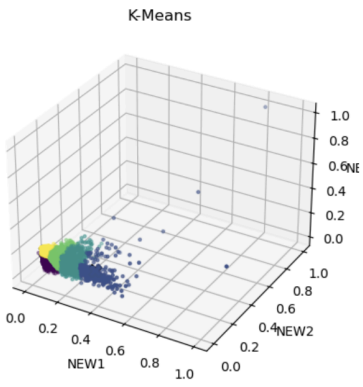
B. In the Bisecting K-means implementation, I utilized the K-means function created from before, calling it with a single cluster rather than the entire data set, with k=2. This means that the k-means function will split this cluster into two new clusters. With Bisecting K-means, we initialize the first cluster as all data points, and then we iteratively split the largest cluster in the data using the method I mentioned before until we have k clusters. We still use the same normalized PCA data, and turn it into a NumPy array. Then, we initialize the starting cluster as the entire data set. I then enter the iteration step, where we get the largest cluster by number of points and run K-means with k=2. I then create two new index sets for the NumPy array, for the child0 and child1 clusters we get from K-means. After splitting the points of the old cluster to their respective new clusters, I replace the old cluster with child0 and append child1to the set. This process repeats until we have k clusters, where I then create an array to assign each data point to its respective cluster index, and get the centroids by taking the mean of all points in each cluster.

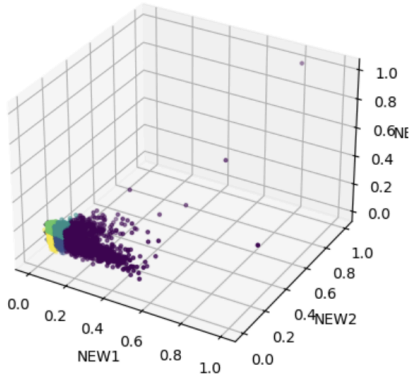
C. In the sse function, we compute the sum of squared errors to find the tightness or quality of each cluster. I first convert each input to a NumPy array, then loop over each cluster, calculating the sse based on the formula. To do this, I find the distance from each point to the centroid within the cluster, then square this distance, and sum it over every point. The lower the sse, the tighter the cluster.

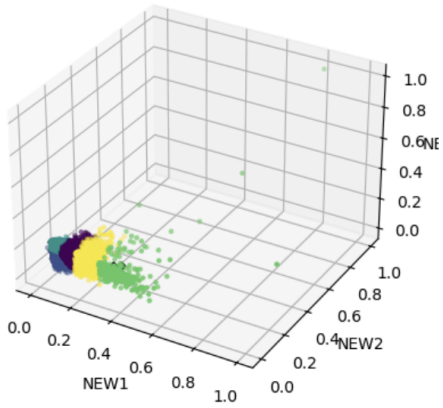
Task 3: Cluster Analysis

A. For this task, I created three scripts, one that ran my K-means implementation 20 times with k=5, one that ran my bisecting K-means implementation 5 times with k=5, and one that ran the Sklearn K-means implementation once. Each script runs the specified implementation the specified number of times, while keeping track of the best run, based on the sum of squared errors. After each run, the centroids are printed and at the end the best sse is also printed. After the specified runs have been completed, I used Matplotlib to create a 3D graph based on the data from the run with the best sse.

B. My K-means: `28.875118042950483`
Bisecting K-means: `38.172784960877934`
Sklearn K-means: `28.875994542723`







Based on the sse's given above, my K-means implementation and the Sklearn implementation ended with very similar results, with Sklearn slightly outperforming my implementation. This could be explained due to the more uniform distribution of starting centroids used rather than my completely random implementation. I would have expected the Sklearn implementation to do far better than mine, but the sse from my implementation was taken from twenty trials whereas the Sklearn implementation only had one attempt to get its sse. The Bisecting K-means implementation was worse than the other two by a larger margin, possibly due to the difference in logic causing for less compact clusters, maybe the splits have more variance to them.

C. We can draw many conclusions from the clustering of the data. I first added the attribute cluster to the data set, to show which cluster each data point was in. I then used the groupby function to show trends within the data, starting with default. Here I saw that cluster 3 had the highest default percentage at 24.91%, and cluster 1 had the lowest at 17.11%.

```
CLUSTER
0    0.194855
1    0.171138
2    0.202432
3    0.249054
4    0.233383
Name: DEFAULT, dtype: float64
```

I then used groupby to get the numeric attribute trends. Here I can see that cluster 1 has the highest age at 43.53, and cluster 3 has the lowest age at 28.61.
Limit Balance Highest: cluster 0, 272246.51
Limit Balance Lowest: cluster 3, 9819063
Bill Amounts Highest: cluster 0
Bill Amounts Lowest: cluster 3
Pay Amounts Highest: cluster 0
Pay Amounts Lowest: cluster 3

CLUSTER	AGE	LIMIT_BAL	BILL_AMT1	BILL_AMT2	BILL_AMT3	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5	PAY_AMT6
0	36.868125	272246.505027	186711.681845	183732.446777	179307.892667	166699.217031	156455.546422	150636.437907	15938.715257	17028.861325	14524.002070	13203.484624	12875.769072	13637.489947
1	43.527290	227830.712303	65936.710453	60514.883441	55697.293710	50184.265495	46507.218316	46688.557817	7269.320074	7725.084644	7133.209528	7293.555042	9174.578631	10783.277521
2	30.003951	139075.987842	67913.617629	66883.370517	64360.344073	59677.117021	56083.003951	53613.218541	7451.441945	7711.937386	7041.481459	6712.201216	5097.867781	5129.969605
3	28.611164	98190.633869	16732.533113	15527.059839	14227.406220	12718.989357	11681.067763	10998.936968	2731.522824	2800.106315	2399.348983	2165.200922	2036.965941	2219.694655
4	43.312668	194219.970194	12926.239642	10977.322057	9837.363189	8694.897615	7828.368852	7588.283308	2618.857377	2676.775410	2389.204918	2250.123845	2433.155738	2992.860507

I then moved onto the categorical attributes where I found that clusters 2 and 3 have around 50% college education, cluster 4 has 24% with high school education, and cluster 0 has the least amount of education.

```
CLUSTER  EDUCATION
0        2            0.467179
         1            0.376404
         3            0.131579
         5            0.016558
         4            0.004731
         6            0.003548
1        2            0.434783
         1            0.315449
         3            0.232192
         5            0.009713
         4            0.004163
         6            0.002775
         0            0.000925
2        2            0.520973
         1            0.348632
         3            0.114894
         5            0.010638
         4            0.003951
         6            0.000608
         0            0.000304
3        2            0.505322
         1            0.361637
         3            0.121570
         5            0.006859
         4            0.003903
         6            0.000473
         0            0.000237
4        2            0.405365
         1            0.341282
         3            0.239195
         5            0.007004
         4            0.004620
         6            0.001639
         0            0.000894
Name: proportion, dtype: float64
```

With marriage, I found that clusters 2 and 3 are mostly not married, clusters 1 and 4 are mostly married, and cluster 0 has a relatively even split.

```
CLUSTER  MARRIAGE
0        2           0.499113
         1           0.494973
         3           0.005618
         0           0.000296
1        1           0.675301
         2           0.307123
         3           0.015726
         0           0.001850
2        2           0.683283
         1           0.310334
         3           0.004559
         0           0.001824
3        2           0.731079
         1           0.260407
         3           0.006623
         0           0.001892
4        1           0.670492
         2           0.305514
         3           0.021311
         0           0.002683
Name: proportion, dtype: float64
```

From sex, I found that all clusters have a majority of females, with clutter 2 having the highest percent at 64%, and cluster 1 having the lowest percent at 56%.

```
CLUSTER  SEX
0        2      0.583974
         1      0.416026
1        2      0.560592
         1      0.439408
2        2      0.642857
         1      0.357143
3        2      0.615184
         1      0.384816
4        2      0.597317
         1      0.402683
Name: proportion, dtype: float64
```

From the Gini impurity of the default attribute, I was able to find that cluster 1 has the lowest value at 0.28, meaning they are at the lowest risk of defaulting, whereas cluster 3 has the highest at 0.37.

```
CLUSTER
0    0.313773
1    0.283699
2    0.322906
3    0.374052
4    0.357831
Name: DEFAULT, dtype: float64
```

D.